

System Design Interview Guide for Software Architecture - GoodNotes Detailed Notes

PAGE 1: Introduction

Video Focus

- How to approach system design interviews with a repeatable framework.
- Why interviewers evaluate trade-offs, not just a perfect final architecture.

Core Concepts

- **Clarify the problem first:** users, traffic, read/write ratio, latency goals, and consistency needs.

Structure answers in layers:

- Requirements (functional + non-functional)
- High-level design
- Deep dives (data model, scaling, reliability)
- Bottlenecks and trade-offs
- Communicate assumptions explicitly to avoid misalignment.

Worked Example

- **Prompt:** "Design a scalable URL shortener."

Good opening sequence:

- Ask expected QPS, retention, custom aliases, analytics requirements.
- Define SLA targets (e.g., p95 latency, availability).
- Start with API + storage + cache + ID generation.

Architecture Snapshot

`text

```
Client -> API Gateway -> App Service -> Cache -> DB
      \-> Queue -> Analytics Pipeline
...

```

Interview-Ready Flashcards

Q: What is the first thing to do in a system design interview?

A: Clarify scope and constraints before proposing architecture.

Q: Why are assumptions important?

A: They define boundaries and justify design trade-offs.

Q: What do interviewers look for most?

A: Structured thinking, prioritization, and trade-off awareness.

30-Day Retention Bullets

- Start every answer with requirements and constraints.
- **Use a layered explanation:** broad to deep.
- **State trade-offs early:** consistency, latency, cost, complexity.

PAGE 2: Choosing the Right DB

Video Focus

- Selecting between SQL and NoSQL based on workload and access patterns.
- Matching database choice to consistency, scaling, and query flexibility requirements.

Core Concepts

SQL is strong for:

- ACID transactions
- relational joins
- strict schema and integrity

NoSQL is strong for:

- horizontal scale
- flexible schema
- high write throughput

Common decision filters:

- Read-heavy vs write-heavy
- Strong consistency vs eventual consistency
- Query complexity vs simple key-based access

Worked Example

Ride-sharing payments ledger:

- Use relational DB for transactions and financial correctness.

Session store / profile cache:

- Use key-value/document DB for low latency at scale.

Architecture Snapshot

`text

```
Transactional path -> SQL primary + replicas
High-throughput path -> NoSQL partitioned cluster
Search path -> Search index
```
```

## Interview-Ready Flashcards

Q: When should SQL be preferred?

A: When transactional integrity and relational queries are critical.

Q: When should NoSQL be preferred?

A: When scale, flexible schema, and low-latency key-based access dominate.

Q: Can both be used together?

A: Yes, polyglot persistence is common in real systems.

## 30-Day Retention Bullets

- Choose DB based on access pattern, not trend.
- Mention partitioning, replication, and consistency mode.
- Tie DB choice directly to business constraints.

## PAGE 3: URL Shortener System Design

## Video Focus

- Designing tiny URL creation and redirection at internet scale.

## Core Concepts

### Functional requirements:

- Create short URL
- Redirect short URL to long URL
- Optional custom alias + expiration + analytics

### Key components:

- API service
- ID generation / hash
- Key-value store
- cache layer
- rate limiter

### Capacity framing:

- Very read-heavy workload (redirects >> creates)

## Worked Example

### Write path:

- Validate URL
- Generate short key
- Persist mapping
- Return short link

### Read path:

- Check cache
- Fallback to DB
- Return 301/302 redirect

## Architecture Snapshot

`text

```
Create API -> ID Generator -> KV Store
Read API -> Cache (hit?) -> KV Store -> Redirect
...
```

## Interview-Ready Flashcards

Q: Why cache is critical here?

A: Redirect traffic is high and repetitive, so cache cuts DB load and latency.

Q: How to avoid key collisions?

A: Use deterministic namespace or globally unique ID strategy.

Q: What HTTP response is used for redirect?

A: Typically 301 or 302 depending on permanence.

### 30-Day Retention Bullets

- State read-heavy nature and cache-first strategy.
- Discuss key generation and collision handling.
- **Include abuse controls:** rate limits and malicious URL checks.

## PAGE 4: Airbnb / Booking.com System Design

### Video Focus

- Building a large-scale accommodation marketplace with search and booking consistency.

### Core Concepts

#### Two distinct workloads:

- Search and discovery (read-heavy, geo filters, ranking)
- Booking and payment (transaction-heavy, consistency-critical)

#### Avoid double booking with:

- inventory locking
- transactional confirmation
- idempotent booking APIs

#### Search optimization:

- denormalized listing index
- geo indexing
- precomputed ranking signals

## Worked Example

### Booking flow:

- User selects room/date
- System checks availability lock
- Payment authorization
- Booking confirmation + event publish

## Architecture Snapshot

`text

```
User -> Search Service -> Search Index
User -> Booking Service -> Inventory DB -> Payment
 \-> Event Bus -> Notifications
...

```

## Interview-Ready Flashcards

Q: Why separate search and booking storage models?

A: Search needs fast denormalized reads; booking needs strict consistency.

Q: How is overselling prevented?

A: Locking or reservation hold with transactional commit.

Q: Why idempotency keys?

A: To prevent duplicate charges/bookings on retries.

## 30-Day Retention Bullets

- Separate discovery from transaction path.
- Prioritize consistency for inventory updates.
- Include cancellation/refund flows in edge cases.

## PAGE 5: Amazon System Design

## Video Focus

- Designing an e-commerce platform with catalog, cart, checkout, and order lifecycle.

## Core Concepts

### Core domains:

- Product catalog
- Search
- Cart
- Orders
- Payments
- Inventory

### Design principles:

- service decomposition by domain
- asynchronous events for order pipeline
- eventual consistency across non-critical joins

### Hot paths:

- Product page read latency
- Checkout reliability

## Worked Example

### Checkout path:

- Validate cart and stock
- Create order record
- Process payment
- Reserve/decrement inventory
- Emit order events for shipment and notifications

## Architecture Snapshot

`text

```
Catalog -> Search Index
Cart -> Checkout -> Order Service -> Payment
 \-> Inventory
 \-> Event Bus -> Fulfillment
...
```

## Interview-Ready Flashcards

Q: Why event-driven order pipeline?

A: It decouples services and improves resiliency at scale.

Q: Where is strong consistency needed most?

A: Payment and inventory mutation boundaries.

Q: What improves product read speed?

A: CDN + cache + read replicas/indexing.

### 30-Day Retention Bullets

- Model by bounded context (catalog/order/payment).
- Mention idempotent checkout and retry safety.
- Explain where eventual consistency is acceptable.

## PAGE 6: WhatsApp System Design

### Video Focus

- Building low-latency, reliable, real-time messaging at global scale.

### Core Concepts

#### Messaging essentials:

- Persistent connection (WebSocket-like)
- Message queueing and fanout
- Delivery acknowledgments
- Presence and last-seen

#### Reliability concerns:

- Offline message storage
- exactly-once illusions via idempotency and dedupe
- ordered delivery within a chat

#### Scale:

- shard by user/chat ID
- regional routing



## Worked Example

### Send message:

- Sender -> chat gateway
- Persist message
- Push to recipient if online
- Store for later delivery if offline
- **Ack state transitions:** sent, delivered, read

## Architecture Snapshot

`text

```
Client <-> Realtime Gateway -> Chat Service -> Message Store
 \-> Queue -> Push/Notification
...

```

## Interview-Ready Flashcards

Q: Why long-lived connections?

A: They reduce handshake overhead and enable real-time delivery.

Q: How to support offline users?

A: Persist undelivered messages and replay on reconnect.

Q: How is ordering maintained?

A: Partition by conversation and sequence messages.

## 30-Day Retention Bullets

- Mention connection management and heartbeat.
- Explain message states and ack pipeline.
- Cover dedupe/idempotency for retries.

## PAGE 7: Notification System at Scale

## Video Focus

- Designing a multi-channel notification platform (push, email, SMS, in-app).

## Core Concepts

### Requirements:

- fanout to many users
- template rendering
- channel preferences
- throttling and retries

### Core architecture:

- producer services publish events
- notification orchestrator decides channel
- channel-specific workers send notifications

### Reliability:

- dead-letter queues
- backoff retry
- idempotent sends

## Worked Example

- **Event:** order shipped
  - Event bus receives event
  - Rules engine selects channels
  - Templating service personalizes content
  - workers send and update delivery logs

## Architecture Snapshot

`text

**Producer -> Event Bus -> Notification Orchestrator**

-> Template Service

-> Channel Workers (Push/Email/SMS)

...

## Interview-Ready Flashcards

Q: Why separate channel workers?

A: Each channel has different rate limits and failure patterns.

Q: What is DLQ used for?

A: Capturing repeatedly failed messages for inspection/replay.

Q: Why user preference service?

A: To honor opt-in/opt-out and avoid spam.

### 30-Day Retention Bullets

- Separate orchestration from channel delivery.
- Include retries with backoff and DLQ.
- Track delivery analytics and suppression rules.

## PAGE 8: Uber System Design

### Video Focus

- Real-time matching of riders and drivers with geo-distributed low latency.

### Core Concepts

#### Core flows:

- Rider requests trip
- Driver discovery by geospatial proximity
- Dispatch and acceptance
- Trip tracking and billing

#### Data/compute concerns:

- High-frequency location updates
- geospatial index (grid/geohash)
- surge pricing computation

### Worked Example

#### Matching process:

- Rider request enters dispatch service
- Nearby drivers queried from geo index
- candidates ranked by ETA and acceptance score

- first accepted driver gets assignment

## Architecture Snapshot

`text

```
Rider App -> Dispatch -> Geo Index -> Driver Candidates
Driver App -> Realtime Location Stream -> Location Store
...`
```

## Interview-Ready Flashcards

Q: Why geospatial indexing?

A: To quickly find nearest drivers from massive location streams.

Q: What keeps ETA accurate?

A: Frequent location updates plus traffic-aware routing.

Q: Why asynchronous trip events?

A: To decouple billing, receipts, and analytics from dispatch path.

## 30-Day Retention Bullets

- Emphasize low-latency matching loop.
- Include live location ingestion strategy.
- Mention cancellation and re-dispatch edge cases.

## PAGE 9: Twitter System Design

### Video Focus

- Designing a social feed system with high write volume and high read fanout.

### Core Concepts

#### Core features:

- Post tweet
- Follow/unfollow
- Home timeline

- likes/retweets/replies

#### Feed strategies:

- Fanout-on-write (push to followers)
- Fanout-on-read (pull during timeline request)
- Hybrid for celebrity users

#### Storage:

- tweet object store + index
- relationship graph store

### Worked Example

#### Hybrid timeline:

- Normal accounts use fanout-on-write
- Celebrity accounts computed on read to avoid write explosion

### Architecture Snapshot

`text

```
Tweet Service -> Timeline Fanout Workers -> Timeline Store
Follow Graph -> Read API -> Timeline Merge Service
```
```

Interview-Ready Flashcards

Q: Why not pure fanout-on-write always?

A: Celebrity users create massive follower write amplification.

Q: Why cache timelines?

A: Home feed is frequently read and benefits heavily from caching.

Q: What is key trade-off?

A: Write cost vs read latency.

30-Day Retention Bullets

- Explain hybrid timeline strategy clearly.
- Mention graph scale and partitioning.

- Include ranking and relevance pipeline.

PAGE 10: Facebook / Instagram System Design

Video Focus

- Building media-heavy social platforms with feed ranking and interaction scale.

Core Concepts

Major workloads:

- media upload and processing
- feed generation and ranking
- engagement graph updates

Media pipeline:

- upload -> object storage -> transcoding -> CDN delivery

Feed pipeline:

- candidate generation
- ranking model
- personalization and freshness constraints

Worked Example

Photo upload flow:

- User uploads image
- metadata stored
- async resize/transcode jobs
- CDN invalidation/update
- feed event emitted

Architecture Snapshot

`text

```
Upload API -> Object Store -> Media Processing -> CDN
Post/Event -> Feed Candidate Service -> Ranker -> Home Feed API
...
```

Interview-Ready Flashcards

Q: Why async media processing?

A: Heavy transforms should not block user upload response.

Q: Why CDN is essential?

A: Global low-latency media delivery and origin offload.

Q: What feed quality dimensions matter?

A: Relevance, freshness, diversity, and safety.

30-Day Retention Bullets

- Separate media serving from feed ranking concerns.
- Mention object storage + CDN pattern.
- Include abuse moderation checkpoints.

PAGE 11: YouTube / Netflix System Design

Video Focus

- Designing large-scale video platforms for upload, processing, and streaming.

Core Concepts

End-to-end flow:

- upload ingestion
- transcoding to multiple bitrates/resolutions
- chunked distribution via CDN
- adaptive bitrate playback

Metadata and discovery:

- video catalog
- search and recommendations

Reliability:

- multi-region replication
- hot content caching

Worked Example

Playback path:

- Client requests manifest
- player selects bitrate based on network
- pulls video chunks from nearest CDN edge

Architecture Snapshot

`text

```
Uploader -> Ingestion -> Transcoding Farm -> Origin Storage -> CDN
Viewer -> Playback API -> Manifest -> CDN Segments
```
```

## Interview-Ready Flashcards

Q: Why adaptive bitrate streaming?

A: It keeps playback smooth across changing network conditions.

Q: Why pre-transcode multiple versions?

A: Different devices and bandwidth need different encodings.

Q: What is CDN role here?

A: Serve high-volume segments close to users.

## 30-Day Retention Bullets

- Mention transcode pipeline and storage lifecycle.
- Explain manifest + chunk-based delivery.
- Discuss recommendation and watch-history scale.

## PAGE 12: Zoom System Design

### Video Focus

- Building low-latency video conferencing with reliable audio/video quality.

### Core Concepts



### Meeting architecture:

- signaling plane for session setup
- media plane for RTP/stream transport
- SFU/MCU options for multiparty calls

### QoS factors:

- jitter buffering
- packet loss handling
- adaptive bitrate

### Collaboration features:

- screen sharing
- chat
- recording

### Worked Example

#### Group meeting:

- participants connect to nearest regional media node
- SFU forwards selected streams
- dynamic adaptation adjusts quality per participant network

### Architecture Snapshot

`text

```
Client -> Signaling Service -> Session Control
Client <-> Media Node (SFU) <-> Other Participants
```
```

Interview-Ready Flashcards

Q: Why SFU is common for large calls?

A: It scales better than full mesh by forwarding streams efficiently.

Q: What does signaling handle?

A: Session setup, participant state, and control messages.

Q: How to reduce perceived lag?

A: Regional edge nodes plus adaptive media strategies.

30-Day Retention Bullets

- Separate signaling and media planes.
- Include packet loss/jitter mitigation.
- Discuss scaling strategy by meeting size.

PAGE 13: Google Maps System Design

Video Focus

- Designing map rendering, routing, and location search at global scale.

Core Concepts

Core capabilities:

- map tile serving
- geocoding and reverse geocoding
- routing and ETA
- traffic updates

Data strategy:

- partition by geo tiles
- cache hot regions aggressively
- periodic map data refresh pipelines

Routing complexity:

- shortest path + traffic constraints
- multi-criteria optimization

Worked Example

Route request:

- Source/destination geocoded
- route engine computes candidate paths
- traffic model adjusts ETA
- turn-by-turn response returned

Architecture Snapshot

`text

```
Client -> Maps API -> Tile Service + Geocode Service + Route Engine
Traffic Stream -> Realtime Processing -> ETA Model
```
```

## Interview-Ready Flashcards

Q: Why tile-based maps?

A: Tiles enable scalable caching and incremental rendering.

Q: Why separate geocoding service?

A: Address parsing/search has distinct indexing needs.

Q: What drives ETA quality?

A: Historical traffic + realtime signals + route model tuning.

## 30-Day Retention Bullets

- Explain tile caching and geo partitioning.
- Mention realtime traffic ingestion.
- Include fallback when traffic feed is delayed.

## PAGE 14: Biggest Mistakes to Avoid in the Interview

### Video Focus

- Avoiding common system design interview failures.

### Core Concepts

#### Frequent mistakes:

- jumping into components without clarifying requirements
- ignoring scale estimates and bottlenecks
- proposing one-size-fits-all architecture
- not discussing trade-offs
- poor communication and no structured flow

### Better strategy:

- ask targeted clarifying questions
- estimate capacity quickly
- iterate design in layers
- call out alternatives and why you choose one

### Worked Example

- **Weak answer:** "Use microservices + Kafka + Redis + NoSQL everywhere."

### Strong answer:

- "Given read-heavy traffic and moderate consistency needs, we use cache-first reads and partitioned storage; for payment we keep strict transactional boundaries."

### Architecture Snapshot

`text

Requirements -> Capacity -> High-Level Design -> Deep Dive -> Trade-offs -> Risks  
...

### Interview-Ready Flashcards

Q: What is the #1 mistake?

A: Not clarifying requirements and assumptions first.

Q: What separates strong from average answers?

A: Explicit trade-offs and bottleneck-driven refinements.

Q: Should one architecture fit all systems?

A: No, architecture must match workload and constraints.

### 30-Day Retention Bullets

- Keep a consistent answer template under time pressure.
- Narrate design decisions with trade-offs.
- Always finish with scaling, reliability, and failure handling.

---

## Final Revision Sheet (1-Minute)

- Start with requirement clarification and scale assumptions.
- Pick storage, cache, and communication patterns by workload.
- Distinguish consistency-critical paths from eventually consistent paths.
- Use async pipelines to decouple heavy/non-blocking work.
- Close every answer with bottlenecks, trade-offs, and improvements.